CS6310 – Software Architecture & Design
Assignment #3 (100 points): Pokémon Tournament Project – Phase 3 (v0)
"Welcome to the Poke-Dome!"
Spring 2025 Semester – created by the CS6310 Instructional Team

Overall Instructions & Submission Deliverables

- This assignment must be completed as part of an approved group.
- You must submit the following items in Canvas:

[1] Functioning Application (50 points): A *functioning version of the application* in a format agreed upon by your evaluating TA. Your application must also include a basic Graphical User Interface (GUI) - details shown below. All source code (Java and otherwise) and links to external packages and other resources used to create your application in a file named ***source_code.zip***

[2] Design Artifacts (40 points): The updated or newly added; "as-is" UML Class, Sequence, Deployment and Use Case ***Design Diagrams***, and any other accompanying documentation, in a file named *design_docs.pdf*

[3] Demonstration Video (10 points): An *MP4 video* (or other comparable format) with a ***total viewing time of 15-20 minutes*** or less named ***ptp_video.mp4*** that presents an overview of your application. You are also welcomed to divide your work into smaller videos to keep the file sizes more reasonable.

- You may divide your class model diagram or sequence diagrams into multiple files if needed. For example, if the initial diagram is so large that dividing it into multiples makes it significantly easier to read, then you may submit multiple files with reasonable name extensions such as *class_model_diagram1.pdf*, *class_model_diagram2.pdf*, etc. You must provide enough context for the multiple diagrams to allow us to connect them together to understand the singular/completed diagram.
- Formats other than PDF must be pre-approved by an instructor or TA.
- You must notify us via a private post on Canvas and/or Ed Discussion BEFORE the Due Date if you are encountering difficulty submitting your project. You will not be penalized for situations where Canvas is encountering significant technical problems. However, you must alert us before the due date – not well after the fact. You are responsible for submitting your answers on time in all other cases.
- For this project, you should submit your materials via Canvas. The video can sometimes be exceptionally large, so a link to an external but accessible storage site (e.g., One Drive) should be submitted via Canvas for the movie.
- Uploading files to Canvas occasionally takes a significantly long time, especially based on the relative size of the files being submitted. Please plan accordingly, as submissions outside of the Canvas Availability Date will likely not be accepted.
- You are also permitted – and highly recommended - to take advantage of the opportunity to submit your files an unlimited number of times. Use the same file naming standards for your (optional) "interim submissions" that you do for the final submission.

[1] Functioning Application (50 points): You must submit a working application that satisfies the client's requirements as presented throughout all the assignments, and as clarified further through all associated Office Hours and Piazza posts. You must also provide copies of the actual source code that your team has developed, along with references to all the external packages, libraries, frameworks, services and systems used to develop your application.

Your system must incorporate all the design requests as described in all the previous assignments. From a grading perspective, the points for implementation are distributed approximately as follows:

- 10 points for the clarity and "ease-of-use" (whether it's a GUI or a CLI) of your system, as well as the extent to which the interface supports the normal flow of operations during each simulation run.
- 10 points for the overall correctness of your monitoring system to include the accuracy of the simulation state.
- 20 points for the correctness of all your proposed modifications.
- 10 points for the overall architectural and design quality of your system, including (but not limited to) factors such as good use of abstraction, modular design, reasonable documentation, and descriptive class, variable and method names.

You are allowed to submit your application in a format that has received prior approval from your evaluating TA.

[2] Design Artifacts (40 points): You have already developed numerous design documents: Class Diagrams, Sequence Diagrams, Object Diagrams, and possibly other forms such as State Charts and/or Collaboration Diagrams. For this assignment, you must select and provide the most appropriate UML-compliant Structural and Behavioral Diagrams that describe the final design of your application.

Please clearly designate which version of UML you will be using for consistency. We highly recommend version 2.0 or later (preferable, up through the latest OMG-accepted version: 2.5 at https://www.iso.org/standard/52854.html) as opposed to version 1.4 or earlier. There are significant differences between the versions, so your diagrams must be consistent with the specific version that you've designated.

From a grading perspective, the points for documentation are distributed as follows:

- 16 points for your UML Structural Diagrams, which should include (at a minimum):
  - A Class (Model) Diagram that describes your Java source code system as it is implemented
  - A Deployment (or similar) Diagram that shows how your system is implemented, especially in integrating external packages, services and/or systems
- 16 points for your UML Behavioral Diagrams which should include (at a minimum):
  - Sequence, State Machine (or similar) Diagrams the give a reasonable overview of how the system is implemented

    - Use Case Diagrams for all functionality added per your design proposals
- 8 points for the overall quality and presentation of your design documentation, including the addition of various diagrams that add significant value for the clients in understanding and maintaining your system into the future.

Do not simply create lots of diagrams to attempt to collect points – this is definitely a "quality over quantity" issue. Ensure that new diagrams and/or documents are well formatted and add significant value. And review the Larman text or other online sources for examples of how these diagrams should be represented in terms of the syntax, symbols, etc.

Your Class (Model) Diagram must cover the entirety of your Java source code. By the same token, your Sequence Diagram(s) and Use Case Diagram(s) must focus on the specific functional and non-functional additions that you've made outside of the required core functionality.

There are no more pending or additional requirements after this assignment, so your final design documents should be as accurate and consistent with the final implementation of your application as possible. And especially since you have permission to use languages and system stacks other than Java, you need to ensure that your design documentation is consistent with your actual implementation.

[3] Demonstration Video (10 points): Your team must submit a video demonstration of the prototype that your all have developed. You can either submit a YouTube link (preferred) or an MP4 attachment in Canvas. If submitting a video attachment, try to ensure that the total size doesn't exceed ~200MB, which should cover approximately 20 minutes for a 720p (HD) with 1 Mbps bitrate (or 24fps) quality video. You are also welcomed to divide your work into smaller videos to keep the file sizes more reasonable.

Please take these values as approximations – the real file sizes will depend greatly on other factors such as frame rates, encoding types, etc. Also, we realize that not everyone wants to "make it to the big screen": faces are welcome, but there's no requirement for your faces to be displayed during the video. A screen capture of your system in action, along with a clear, audible English-language audio track, will be sufficient.

Your video should be organized to give a clear demonstration of your system in action, including a well-considered test case/scenario that allows you to show how your interface and supporting components operate; and, how your overall system meets the client's requirements. And your video should highlight your design modifications. There's no need to spend significant amounts of time demonstrating the capabilities that already existed in the system – focus on the aspects of your system that have been changed, upgraded, etc.

Writing Style Guidelines
The style guidelines can be found on the course Udacity site, and at:
https://s3.amazonaws.com/content.udacity-data.com/courses/gt-cs6310/assignments/writing.html

The deliverables should be submitted in the appropriate formats (PDF, ZIP, JAR, etc.) with file names such as ***source_code.zip***, ***design_docs.pdf*** and ***ptp_video.mp4***. You should also use a similarly clear and simple name structure if you need to submit a file that we haven't explicitly listed here. Ensure that all files are clear and legible; points will likely be deducted for unreadable submissions.

Sample Code
- We've included some sample Java source "starter code" that provides the command loop for terminal-based data input. We've also provided some test cases, though you are also welcomed (and highly encouraged) to develop your own tests as well.
- You are permitted to share your test cases with fellow students, but you are not permitted to share code and/or specific discussions or directions on the code or designs need to pass the cases.
- The links to the GitHub repositories will be shared on ed discussions.
  - 1 link will be focused on sample code for the Pokémon project
  - 1 link will be focused on how to add certain functional and non-functional aspects to a repository
  - You should investigate ways of combining these repositories and making them your own. Remember your project code should always remain private even after you finish the class or graduate from the program.

Closing Comments & Suggestions
This is the information provided by the customer so far. We (the OMSCS 6310 Team) will conduct Office Hours where you will be permitted to ask us questions to clarify the client's intent, etc. We will answer some of the questions, but we will not necessarily answer all of them. Also, though this current version of the system is the "core" of the system going forward, our clients will add, update, and remove some of the requirements over the span of the course. One of your main tasks will be to ensure that your architectural documents and related artifacts remain consistent with the problem requirements – and with your system implementations – over time.

Quick Reminder on Collaborating with Others
Please use Ed Discussion for your questions and/or comments, and post publicly whenever it is appropriate. If your questions or comments contain information that specifically provides an answer for some part of the assignment, then please make your post private first, and we (the OMSCS 6310 Team) will review it and decide if it is suitable to be shared with the larger class. Best of luck to you with this assignment, and please contact us if you have questions or concerns.